

Contents

1	Python Interview Questions for Freshers	1
1.1	Questions	1
2	Answers	2
2.1	1. What is Python? What are the benefits of using Python?	2
2.1.1	Key Benefits	2
2.1.2	Example	2
2.2	2. What is a dynamically typed language?	2
2.3	3. What is an interpreted language?	3
2.4	4. What is PEP 8 and why is it important?	3
2.5	5. What is scope in Python?	4
2.6	6. What are lists and tuples? What is the key difference between them?	4
2.6.1	List	4
2.6.2	Tuple	4
2.7	7. What are the common built-in data types in Python?	5
2.8	8. What is pass in Python?	5
2.9	9. What are modules and packages in Python?	6
2.10	10. What are global, protected, and private attributes in Python?	6
2.11	11. What is the use of self in Python?	6
2.12	12. What is __init__() ?	7
2.13	13. What are break , continue , and pass in Python?	7
2.14	14. What are unit tests in Python?	8
2.15	15. What is a docstring in Python?	8
2.16	16. What is slicing in Python?	9
2.17	17. How can you make a Python script executable on Unix?	9
2.18	18. What is the difference between Python arrays and lists?	10

1 Python Interview Questions for Freshers

1.1 Questions

1. What is Python? What are the benefits of using Python?
2. What is a dynamically typed language?
3. What is an interpreted language?
4. What is PEP 8 and why is it important?
5. What is scope in Python?
6. What are lists and tuples? What is the key difference between them?
7. What are the common built-in data types in Python?
8. What is **pass** in Python?
9. What are modules and packages in Python?
10. What are global, protected, and private attributes in Python?
11. What is the use of **self** in Python?
12. What is **__init__()**?
13. What are **break**, **continue**, and **pass** in Python?
14. What are unit tests in Python?
15. What is a docstring in Python?

16. What is slicing in Python?
 17. How can you make a Python script executable on Unix?
 18. What is the difference between Python arrays and lists?
-

2 Answers

2.1 1. What is Python? What are the benefits of using Python?

Python is a **high-level, interpreted, general-purpose programming language** known for simple syntax and readability.

2.1.1 Key Benefits

- Easy to learn and read
- Dynamically typed
- Supports object-oriented programming
- Large standard library
- Large third-party ecosystem
- Cross-platform
- Open source
- Useful for web development, automation, data engineering, AI, machine learning, and scripting

2.1.2 Example

```
def greet(name):  
    return f"Hello, {name}"
```

```
print(greet("Alice"))
```

Interview answer: Python is a high-level, interpreted, dynamically typed programming language known for readable syntax, rapid development, and a large ecosystem of libraries.

[Back to Questions](#)

2.2 2. What is a dynamically typed language?

A dynamically typed language determines variable types at **runtime**.

Python does not require explicit variable type declarations.

```
x = 10  
print(type(x))
```

```
x = "Python"  
print(type(x))
```

The same variable can reference objects of different types.

Interview answer: Python is dynamically typed because variable types are determined at runtime rather than being declared in advance.

[Back to Questions](#)

2.3 3. What is an interpreted language?

Python is commonly described as an interpreted language.

In CPython, source code is first converted into **bytecode**, which is then executed by the Python Virtual Machine.

```
Python source (.py)
    ↓
  Bytecode
    ↓
Python Virtual Machine
    ↓
  Execution
```

Interview answer: Python is generally called interpreted, although CPython first compiles source code into bytecode and then executes that bytecode.

[Back to Questions](#)

2.4 4. What is PEP 8 and why is it important?

PEP stands for **Python Enhancement Proposal**.

PEP 8 is Python's official style guide.

It provides conventions for:

- Naming
- Indentation
- Imports
- Whitespace
- Code formatting
- Line length

Example:

```
def calculate_total_price():
    pass
```

PEP 8 improves readability and consistency across Python projects.

Interview answer: PEP 8 is Python's official coding style guide. It defines conventions that make Python code more readable and consistent.

[Back to Questions](#)

2.5 5. What is scope in Python?

Scope defines where a variable can be accessed.

Python follows the **LEGB** rule:

- **L**ocal
- **E**nclosing
- **G**lobal
- **B**uilt-in

Example:

```
x = 10
```

```
def test():  
    y = 20  
    print(x, y)
```

```
test()
```

Here:

- x is global
- y is local

Interview answer: Scope determines where names are accessible. Python resolves names using the LEGB rule: Local, Enclosing, Global, and Built-in.

[Back to Questions](#)

2.6 6. What are lists and tuples? What is the key difference between them?

Lists and tuples are ordered collections.

2.6.1 List

```
numbers = [1, 2, 3]
```

Lists are **mutable**.

2.6.2 Tuple

```
numbers = (1, 2, 3)
```

Tuples are **immutable**.

List	Tuple
Mutable	Immutable
Uses []	Uses ()

List	Tuple
Can be modified	Cannot be modified
Suitable for changing data	Suitable for fixed data

Interview answer: The key difference is that lists are mutable while tuples are immutable.

[Back to Questions](#)

2.7 7. What are the common built-in data types in Python?

Common Python built-in types include:

Category	Types
Numeric	int, float, complex
Boolean	bool
Text	str
Sequence	list, tuple, range
Mapping	dict
Set	set, frozenset
Null	NoneType

Example:

```
age = 30
name = "Alice"
skills = ["Python", "SQL"]
user = {"name": "Alice", "age": 30}
```

Interview answer: Common built-in Python types include integers, floats, strings, lists, tuples, dictionaries, sets, booleans, ranges, and None.

[Back to Questions](#)

2.8 8. What is pass in Python?

`pass` is a null statement.

It performs no operation.

It is commonly used as a placeholder.

```
def future_function():
    pass
```

Interview answer: `pass` does nothing and is used when Python requires a statement syntactically but no implementation is needed yet.

[Back to Questions](#)

2.9 9. What are modules and packages in Python?

A **module** is typically a Python file containing reusable code.

Example:

```
import math
```

A **package** organizes multiple modules under a common namespace.

Example structure:

```
utilities/  
    __init__.py  
    math_utils.py  
    string_utils.py
```

Interview answer: A module is usually a single .py file, while a package groups related modules together.

[Back to Questions](#)

2.10 10. What are global, protected, and private attributes in Python?

Python does not use strict access modifiers like Java.

Instead, naming conventions are used.

```
class Employee:  
    def __init__(self):  
        self.name = "Alice"  
        self._salary = 50000  
        self.__password = "secret"
```

Meaning:

- `name` → public
- `_salary` → protected/internal-use convention
- `__password` → name-mangled

Double underscores do not provide true security.

Interview answer: Python uses naming conventions instead of strict access modifiers. A single underscore indicates internal use, while double underscores trigger name mangling.

[Back to Questions](#)

2.11 11. What is the use of `self` in Python?

`self` refers to the current instance of a class.

```
class Employee:
    def __init__(self, name):
        self.name = name
```

Here:

```
self.name
```

belongs to the current object.

`self` is a convention, not a Python keyword.

Interview answer: `self` represents the current instance and allows instance methods to access the object's attributes and methods.

[Back to Questions](#)

2.12 12. What is `__init__()`?

`__init__()` initializes an object after it has been created.

```
class Employee:
    def __init__(self, name):
        self.name = name
```

```
employee = Employee("Alice")
```

More precisely:

```
__new__() → creates the object
__init__() → initializes the object
```

Interview answer: `__init__()` is a special method automatically called after object creation and is normally used to initialize instance attributes.

[Back to Questions](#)

2.13 13. What are `break`, `continue`, and `pass` in Python?

Keyword	Purpose
<code>break</code>	Exit the loop
<code>continue</code>	Skip the current iteration
<code>pass</code>	Do nothing

Example:

```
for i in range(5):
    if i == 2:
        continue
```

```
print(i)
```

Output:

```
0
1
3
4
```

Interview answer: `break` exits a loop, `continue` skips the current iteration, and `pass` performs no operation.

[Back to Questions](#)

2.14 14. What are unit tests in Python?

Unit tests test small parts of an application independently.

Common examples include testing:

- Functions
- Methods
- Classes

Python provides the built-in `unittest` framework.

```
import unittest
```

```
class TestMath(unittest.TestCase):
```

```
    def test_addition(self):
        self.assertEqual(2 + 3, 5)
```

Popular Python testing frameworks include:

- `unittest`
- `pytest`

Interview answer: Unit tests verify individual components independently to catch defects and prevent regressions.

[Back to Questions](#)

2.15 15. What is a docstring in Python?

A docstring documents a module, class, function, or method.

```
def add(a, b):
    """Return the sum of two numbers."""
    return a + b
```

A docstring can be accessed using:

```
print(add.__doc__)
```

or:

```
help(add)
```

Interview answer: A docstring is a documentation string associated with Python modules, classes, functions, or methods.

[Back to Questions](#)

2.16 16. What is slicing in Python?

Slicing extracts part of a sequence.

Syntax:

```
sequence[start:stop:step]
```

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
print(numbers[1:4])
```

Output:

```
[2, 3, 4]
```

Reverse a sequence:

```
numbers[::-1]
```

Interview answer: Slicing extracts part of a sequence using `[start:stop:step]`. The start is inclusive and the stop is exclusive.

[Back to Questions](#)

2.17 17. How can you make a Python script executable on Unix?

Add a shebang:

```
#!/usr/bin/env python3
```

Give the file executable permission:

```
chmod +x script.py
```

Then run:

```
./script.py
```

Interview answer: Add a Python shebang, make the file executable with `chmod +x`, and execute it directly.

[Back to Questions](#)

2.18 18. What is the difference between Python arrays and lists?

Python lists are general-purpose containers.

```
values = [1, "Python", 3.14]
```

Typed arrays contain compatible element types.

```
from array import array
```

```
numbers = array("i", [1, 2, 3])
```

List	Array
Can contain different object types	Usually homogeneous
General-purpose	Typed/numerical
Built-in syntax	Requires module/library

For data-science workloads, NumPy arrays are commonly used.

```
import numpy as np
```

```
numbers = np.array([1, 2, 3])
```

Interview answer: Lists are general-purpose containers that can hold mixed object types, while arrays are generally designed for homogeneous typed data.

[Back to Questions](#)